

Apunte de Cátedra – Computación II

Clase 1: Fundamentos del Análisis de Algoritmos

Unidad 1: Conceptos, Objetivos y Técnicas de Diseño

Año 2025 – Modalidad Híbrida
Prof. Marcelo Fabio Roldán
Departamento DACEFyN – Sede Capital – UNLaR

1. ¿Qué es un Algoritmo?

Definición Formal

Un algoritmo es una secuencia finita de instrucciones bien definidas, no ambiguas y ejecutables mecánicamente, que resuelve un problema o realiza una tarea en un tiempo finito.

Esta definición, basada en los fundamentos de la computación teórica, fue formalizada por matemáticos como Alan Turing y Donald Knuth, y es aplicable tanto a procesos manuales como a programas informáticos.

Características Fundamentales (según Knuth)

Un algoritmo válido debe cumplir con cinco propiedades esenciales:

Característica	Descripción
Finitud	Debe terminar después de un número finito de pasos.
Precisión	Cada instrucción debe estar claramente definida, sin ambigüedades.
Entrada definida	Debe tener cero o más entradas bien especificadas.
Salida esperada	Debe producir una o más salidas relacionadas con las entradas.
Efectividad	Cada operación debe ser básica y factible de realizar en tiempo razonable.

Ejemplo: Un algoritmo que busca un número en una lista debe detenerse (finitud), seguir pasos claros (precisión), recibir la lista y el número a buscar (entrada), devolver si fue encontrado (salida) y hacerlo con operaciones realizables (efectividad).

2. Objetivos del Análisis de Algoritmos

El análisis de algoritmos permite evaluar y comparar diferentes soluciones a un mismo problema, con el fin de elegir la más adecuada según criterios como:

-  Eficiencia temporal: Tiempo de ejecución.
-  Eficiencia espacial: Uso de memoria.
-  Optimización de recursos: Energía, ancho de banda, costo computacional.
-  Escalabilidad: Comportamiento ante grandes volúmenes de datos.
-  Correctitud: Garantía de que produce el resultado esperado.

 **Caso real:** En 2012, un error en el algoritmo de *trading* de alta frecuencia de *Knight Capital* generó pérdidas de 440 millones de dólares en 45 minutos. Este caso muestra que un algoritmo mal analizado puede tener consecuencias financieras devastadoras.

3. Algoritmos en la Vida Cotidiana y en la Industria

Los algoritmos no son exclusivos de la computación: están presentes en actividades diarias y en los sistemas que usamos constantemente.

Ejemplos Cotidianos

Actividad	Algoritmo implícito
Cocinar pasta	1. Poner agua a hervir. 2. Agregar pasta. 3. Cocinar 8-10 min. 4. Escurrir.
Sacar dinero del cajero (ATM)	1. Insertar tarjeta. 2. Ingresar PIN. 3. Seleccionar monto. 4. Retirar dinero.

Estos procesos tienen entrada, salida, pasos definidos y condiciones de finalización.

Algoritmos en Empresas Tecnológicas

Empresa	Algoritmo clave	Impacto
Netflix	Recomendación basada en comportamiento	Mantiene 250+ millones de usuarios activos; se estima que evita pérdidas de+1 billón USD/año en cancelaciones.

Empresa	Algoritmo clave	Impacto
Google	PageRank	Clasifica páginas según enlaces entrantes; revolucionó la búsqueda en internet.
Amazon	Predicción de demanda	Anticipa compras antes de que el usuario las haga, optimizando logística.
Uber	Matching y ruteo	Conecta conductores y pasajeros en tiempo real usando algoritmos de optimización.

🌐 Reflexión: Los algoritmos median casi todas nuestras interacciones digitales. Como futuros profesionales, su diseño y análisis tienen un impacto ético, económico y social directo.

4. Clasificación de Algoritmos por Técnica de Diseño

Existen diversas estrategias para diseñar algoritmos eficientes. A continuación, se introducen las dos primeras técnicas clave.

a) Fuerza Bruta (Brute Force)

Consiste en probar todas las posibles soluciones hasta encontrar la correcta.

Características

Ventajas	Desventajas
- Fácil de implementar.	- Muy lento para problemas grandes.
- Siempre encuentra la solución (si existe).	- Alto consumo de recursos.
- No requiere conocimiento profundo del dominio.	- Complejidad exponencial en muchos casos.
- Garantiza optimalidad.	- No escalable.

Ejemplo: Romper una contraseña

- Contraseña de 4 dígitos: $10^4=10,000$ combinaciones → viable.
- Contraseña de 8 caracteres (letras + números): $62^8 \approx 218$ billones de combinaciones.

- A 1 millón de intentos por segundo: 6,900 años.
- Conclusión: La seguridad moderna se basa en hacer que la fuerza bruta sea computacionalmente inviable.

💡 Dato: El *minado de Bitcoin* es una forma de fuerza bruta distribuida: la red consume ~150 TWh/año (más que Argentina) buscando hashes válidos.

b) Divide y Vencerás (Divide and Conquer)

Estrategia que divide el problema en subproblemas más pequeños, los resuelve recursivamente y combina las soluciones.

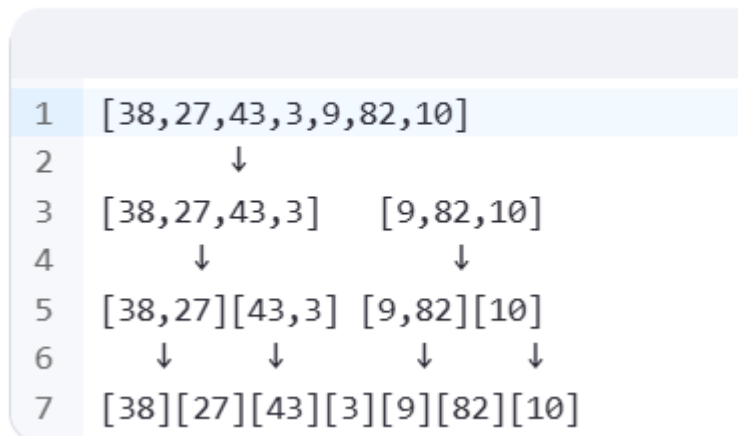
Pasos del Método

1. Dividir: Descomponer el problema en subproblemas.
2. Conquistar: Resolver cada subproblema (recursivamente o directamente si es pequeño).
3. Combinar: Unir las soluciones parciales en una solución global.

Ejemplo: Merge Sort (Ordenamiento por mezcla)

Dado el array: [38, 27, 43, 3, 9, 82, 10]

Fase de División:



Fase de Conquista y Combinación:

1	[27,38][3,43]	[9,82][10]
2	↓	↓
3	[3,27,38,43]	[9,10,82]
4	↓	
5	[3,9,10,27,38,43,82]	

Complejidad

- Tiempo: $O(n \log n)$ en todos los casos (mejor, promedio, peor).
- Espacio: $O(n)$ (requiere memoria adicional).
- Ventaja: Estable (mantiene el orden relativo de elementos iguales).

✦ Aplicaciones: Usado en bases de datos, motores de búsqueda (Google) y como base de Timsort (algoritmo por defecto en Python).

5. Costo de Operaciones Básicas

No todas las operaciones tienen el mismo costo. Entender esto es clave para optimizar algoritmos.

Operaciones de Bajo Costo ($O(1)$)

Operación	Ejemplo
Asignación de variables	<code>x = 5</code>
Operaciones aritméticas	<code>a + b, x * 2</code>
Comparaciones	<code>if x > y</code>
Acceso a array por índice	<code>arr[0]</code>
Llamadas simples a funciones	<code>len(arr)</code>

Operaciones Costosas

--	--	--

Búsqueda lineal	$O(n)$	Recorre todos los elementos
Ordenamiento general	$O(n \log n)$	Merge Sort, Heap Sort
Búsqueda en matriz 2D	$O(n^2)$	Si se recorren todas las celdas
Acceso a disco	Variable	Puede ser 100,000 veces más lento que RAM
Operaciones de red	Alta latencia	50-200 ms por llamada API

❑ Regla de oro: Optimiza primero las operaciones más costosas. Un algoritmo que reduce accesos a disco o red puede ganar más que uno que optimiza operaciones en memoria.

6. Modelos de Cómputo

Para analizar algoritmos, se usan modelos abstractos que simplifican la realidad.

Modelo RAM (Random Access Machine)

- Supuestos:
 - Memoria infinita.
 - Acceso uniforme a cualquier celda en tiempo constante.
 - Operaciones básicas (suma, comparación, etc.) toman 1 unidad de tiempo.
- Uso: Ideal para análisis teórico y cálculo de complejidad asintótica.

Modelo Real (Jerarquía de Memoria)

En sistemas reales, el acceso a memoria no es uniforme:

Nivel	Latencia aproximada	Velocidad relativa
CPU Cache L1	1 ns	1x
Cache L2	3 ns	3x
Cache L3	12 ns	12x
RAM	100 ns	100x

Nivel	Latencia aproximada	Velocidad relativa
Disco SSD	100,000 ns (0.1 ms)	100,000x
Disco HDD	10,000,000 ns (10 ms)	10,000,000x
Red (API)	50,000,000 ns (50 ms)	50,000,000x

💡 Consecuencia: Un algoritmo que accede a datos de forma secuencial (buena localidad espacial) puede ser miles más rápido que uno con accesos aleatorios, aunque tengan la misma complejidad teórica.

7. Actividad Práctica Sugerida

Objetivo: Comparar búsqueda lineal (fuerza bruta) vs. búsqueda binaria (divide y vencerás).

Problema: Buscar un número en una lista de 1,000,000 de elementos.

Aspecto	Búsqueda lineal	Búsqueda binaria
Complejidad	$O(n)$	$O(\log n)$
Peor caso	1,000,000 comparaciones	~20 comparaciones
Requisito	Lista desordenada	Lista ordenada

Tarea:

1. Implementa ambos algoritmos en Python y Haskell.
2. Mide:
 - Tiempo de ejecución (time en Python).
 - Número de comparaciones.
 - Uso de memoria (memory_profiler).
3. Usa datasets de tamaños: 1K, 10K, 100K, 1M.
4. Grafica los resultados.

🛠️ Consejo: Usa cProfile y line_profiler para análisis detallado.

8. Conceptos Clave Aprendidos

- 🔍 Un algoritmo es una secuencia finita, precisa y efectiva de pasos.
- 🔍 La fuerza bruta es simple pero ineficiente para problemas grandes.
- 🔍 Divide y vencerás permite resolver problemas complejos de forma eficiente (ej: Merge Sort).
- 🔍 Las operaciones tienen costos muy distintos: optimiza las más pesadas.
- 🔍 El modelo RAM es útil para teoría, pero el modelo real (con jerarquía de memoria) es clave para optimización práctica.
- 🔍 Los algoritmos impactan directamente en el rendimiento, costo y sostenibilidad de los sistemas.

💬 "En la industria tech, un algoritmo optimizado puede valer millones de dólares."

9. Bibliografía y Recursos Recomendados

Bibliografía Básica

- Paniagua Arís, E. (2003). *Lógica computacional*. Thomson.
⇒ Disponible en: <https://biblioteca.unlar.edu.ar/>

Bibliografía Complementaria

- Cormen, T. H. et al. (2022). *Introduction to Algorithms* (4ª ed.). MIT Press.
→ Capítulos 1, 2 y 3: Fundamentos, crecimiento de funciones, notación asintótica.
- Kleinberg, J. & Tardos, É. (2006). *Algorithm Design*. Pearson.
→ Capítulos 1 y 2: Introducción, análisis asintótico.

Recursos en Línea

- GeeksforGeeks – Analysis of Algorithms
⇒ <https://www.geeksforgeeks.org/analysis-of-algorithms/>
- Khan Academy – Algorithms
⇒ <https://es.khanacademy.org/computing/computer-science/algorithms>
- Haskell MOOC – Functional Programming
⇒ <https://haskellmooc.co.uk/tutorial1>